

STAT 390 - Presentation 2

Team 3

Leah Tan, Xiaolin Liu, Elsa Steen Koppell, Leslie Gonzalez-Flores, and Sofie Langenhuizen

Roles & Responsibilities

Our Team

Leah Consultant

Responsible for suggesting changes and room for improvement in methodology, execution

Sofie Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Xiaolin Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Leslie Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Elsa Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Agenda



Objective



Quick Fixes



Data Augmentation



Hyperparameter Tuning



Attention Scores



Regularization

Agenda



Results



Next Steps



Objective

Modify machine learning model **to more accurately classify** the categories of **C-MIL** (Conjunctival melanocytic intraepithelial lesions)

- Multiple Instance Learning (MIL) across 3 different stains (H&E, Melan-A, Sox10)
- **Test Accuracy: 77.78%, Test Loss: .7272**

Model appears to be underfitting, our method is to promote more aggressive learning

- Trying different base models (DenseNet121 vs. KimiaNet)
- Alter data pre-processing steps and augment it to introduce more diversity
- Tune hyperparameters in training process
- Adjust method to calculate attention scores



Quick Fixes

- Use **KimiaNet** instead of DenseNet in models.py
- Replace RandomResizedCrop() with RandomCrop() in dataset.py > create_transforms()
- Remove saturation jitter in dataset.py > create_transforms()
- Add **persistent_workers=True** when creating the DataLoader in main.py
- Change optimizer from Adam to AdamW in trainer.py
- **Takeaways:** AdamW to speed up convergence and persistent workers to avoid unnecessary deletion of workers in between jobs significantly improved model run time

Test Results:
Test Loss: 0.7272
Test Accuracy: 0.7778



Test Results:
Test Loss: 0.7655
Test Accuracy: 0.7222

Run time 12:33:57,

Run time 04:42:25,



Data Augmentation

Model training parameters result in unused slice data, limiting our data diversity that could improve our model

- The sbatch file currently has parameters: `per_slice_cap 500, max_slices_per_stain 5`
- Benign cases with unused data: 26, 22, 24, 25, 27, 82, 65

Given class imbalance, **split these cases into pseudo-cases to provide more data for training**

data_utils.py

- **`split_cases_with_excess_slices()`**: creates new pseudo-cases, returns a new dictionary to connect patches/slices to case id
- **`extract_original_case_id()`**: extract original case id from pseudo to properly label

main.py

- Added to `prepare_data()` following `group_patches_by_slice()`

Results/Improvement:

Test Loss: 0.9731, Test Accuracy: 0.7778

- Not very influential, likely because not many new cases or misweighting for benign cases

```
Case 26 with stain unmatched has 14 slices and will be split.
Case 65 with stain match has 10 slices and will be split.
Case 27 with stain unmatched has 8 slices and will be split.
Case 24 with stain unmatched has 10 slices and will be split.
Case 22 with stain unmatched has 13 slices and will be split.
Original unique cases: 88
New unique cases: 95
Pseudo-cases added: 7
```



Hyperparameter Tuning Code

- Tuned the following model hyperparameters using **Optuna** for Bayesian tuning:
 - **Dropout:** (0.1, 0.5)
 - **Learning Rate:** (1^{-5} , 1^{-3})
 - **Weight Decay:** (1^{-6} , 1^{-3})
 - **Benign Class Weight:** (1.0, 5.0)
- Created new hyperparameter_tuning.py script
 - Defines **HyperparameterTuner** class
 - **Objective()** method: defines parameter distributions, calculates and returns best validation loss from each trial
 - **Optimize()** method: creates Optuna study to minimize the objective function
 - Using a median pruner and TPESampler
 - **_save_results()** method: returns best parameters and analysis images
 - Defines a **main()** function to execute tuning
- **Takeaway:** Noticed that the original code was using a single dropout hyperparameter for all dropout layers in the model. Different kinds of networks (computing attention scores vs. computing class probabilities) may have different dropout needs, could create and tune a separate hyperparameter for each Dropout occurrence.



Hyperparameter Tuning Results

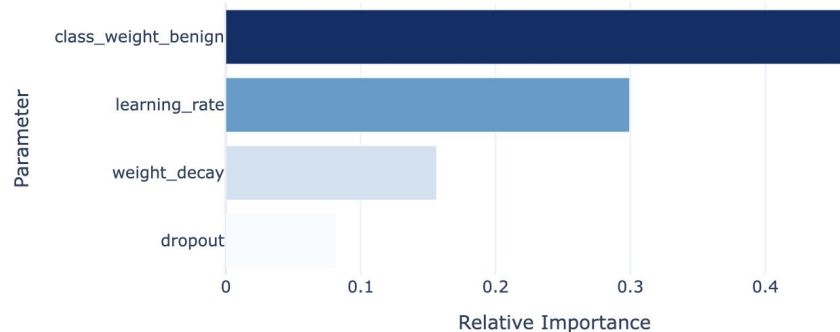
- Original parameter values:
 - **Dropout:** 0.3 (slight increase)
 - **Learning Rate:** $3e-4$ (slight decrease)
 - **Weight Decay:** $2e-4$ (decrease)
 - **Benign Class Weight:** 2.5 (increase)

```
Study name      : mil_tuning_20260212_104753
Trials run      : 8 / 10 (job timed out)
Best trial      : #2
Best val loss   : 0.3781
```

Best hyperparameters:

```
dropout: 0.34044600469728353
learning_rate: 0.0002607024758370766
weight_decay: 1.1527987128232402e-06
batch_size: 1
class_weight_benign: 4.879639408647977
```

Hyperparameter Importances (|Pearson r| with val loss)

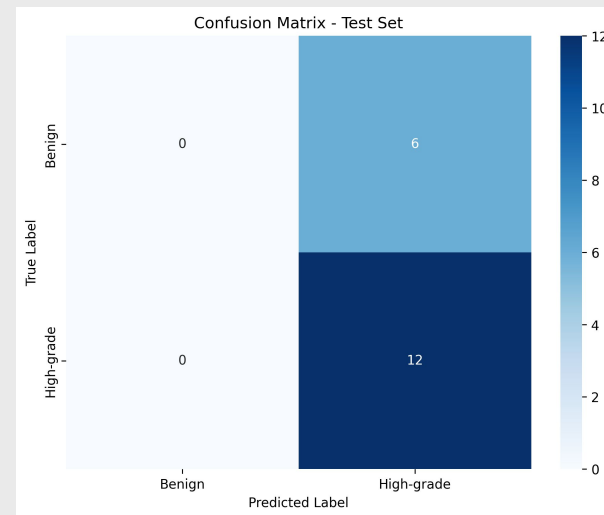




Hyperparameter Tuning Results

- Updated model performance:

```
Test Results:  
Test Loss: 0.6395  
Test Accuracy: 0.6667  
Samples: 18
```



- Takeaways:**
 - Adjusting the hyperparameters resulted in a model that predicted everything as high grade
 - Indicating that we may need more trials (only 8 completed), it may be better to not tune class weights, or tuning may simply not be effective



Hyperparameter Tuning Math

Hyperparameter Tuning Using Optuna Bayesian Optimization

- Main goal: Automate and accelerate machine learning hyperparameter tuning to provide key configurations
- Three components:
 - ◆ **Black box function:** function $f(x)$ want to have minimized. In our case, **validation loss** given a set of hyperparameters.
 - ◆ **Probabilistic Model:** an inexpensive way to predict $f(x)$ across the hyperparameter space, while also providing uncertain estimates for where the model should explore hyperparameter values
 - **Tree-structured Parzen Estimator (TPE)** is default for optuna and used in our model
 - ◆ **Acquisition Function:** decides the next hyperparameter configuration to evaluate



Hyperparameter Tuning Math

Optuna Hyperparameter Workflow

Define Hyperparameter Space
Sample Hyperparameters (First Random, then TPE Algorithm)
Trial Begins
Train Epochs
Validation Curves Produced
Pruning Check
Final Trial Score
TPE Posterior Update
Repeat until All Trials Completed



Hyperparameter Tuning Math

Detailed Workflow Steps (Watanabe)

- **Define Hyperparameter Space:** $X_d \subseteq \mathbb{R}$ (for $d = 1, \dots, D$)
- **n_startup_trials = 5:** For trials 1-5, each hyperparameter is sampled independently from its defined space, with each configuration trained on the model across epoch values.
 - ◆ Builds initially dataset for TPE algorithm: $D := \{(X_n, y_n)\}$ for $n=1, \dots, N$; utilizes best validation loss value from each trial; each trial produces an validation loss curve at each epoch to visually evaluate performance
- **n_warmup_steps = 10 :** Median Pruning (comparing current trial's performance and stops if worse than the median of all previously completed trials) cannot occur before the 10th epoch in a trial
 - ◆ Pruning will not occur in startup trials, but can occur in later ones

Once startup trials are completed, TPE begins



Hyperparameter Tuning Math

TPE (Watanabe)

- **Goal:** Find the best hyperparameter configuration by modeling distributions of **good vs bad** hyperparameters and sampling from promising regions;
- **gamma / threshold:** split threshold for what % of trials are considered "good", in our case, $\gamma=0.25$
- **Splitting:** split dataset D into "good" and "bad" trials utilizing gamma threshold
 - ◆ $D^{(l)}$, $D^{(g)}$, better group and worse group in D
- **Compute weights (importance score):** each past trial has weight computed based on trial ranking
- **Choose bandwidths:** bandwidths for how smooth Kernel Density Functions (KDE) will be
- **Build Density Functions:** build density functions for better and worse group from KDEs *density functions used to approximate next values of hyperparameters*

$$p(\mathbf{x}|y, \mathcal{D}) := \begin{cases} p(\mathbf{x}|\mathcal{D}^{(l)}) & (y \leq y^\gamma) \\ p(\mathbf{x}|\mathcal{D}^{(g)}) & (y > y^\gamma) \end{cases},$$

$$p(\mathbf{x}|\mathcal{D}^{(l)}) = w_0^{(l)} p_0(\mathbf{x}) + \sum_{n=1}^{N^{(l)}} w_n k(\mathbf{x}, \mathbf{x}_n | b^{(l)}),$$

$$p(\mathbf{x}|\mathcal{D}^{(g)}) = w_0^{(g)} p_0(\mathbf{x}) + \sum_{n=N^{(l)}+1}^N w_n k(\mathbf{x}, \mathbf{x}_n | b^{(g)})$$



Hyperparameter Tuning Math

TPE Continued (Watanabe)

- Sample S number of candidate hyperparameters from $p(x|D^{(0)})$; choosing candidates from promising regions
- **Compute Density Ratio:**
 - ◆ $l(x)$ = density of "good" hyperparameters
 - ◆ $g(x)$ = density of "bad" hyperparameters
 - ◆ $r(x) = l(x) / g(x)$ **Acquisition Function**
- Run model with chosen hyperparameters from x^*
- Add result from x^* to dataset, D
- **Loop Continues until all trials have finished**
- **Best hyperparameters configuration is what provide best performance of minimized validation loss**

$$\text{Pick } x_{N+1} := x^* \in \operatorname{argmax}_{x \in \mathcal{S}} r(x|\mathcal{D})$$

Takeaway/Improvement: If continue tuning with Optuna, first need to increase number of overall trials to reduce randomness (currently at 10 trials, with 5 as startup trials). Second, increase number of `n_warmup_steps` to prevent pruning too early and cutting off epochs in trials early before the model has had a chance to learn. Lastly, increase number of epochs per trial to be greater than 20. However, are computationally expensive and can drastically increase runtime.



Optuna in Research Papers

Deep Transfer Learning Based Models with Optuna Hyperparameter Optimization Colorectal Cancer Images

Classification (**Wasito and Saptono**)

- ◆ 5 Deep Learning Models Tuned with Series of Tuning Methods
 - Models: Efficient Net B2, ResNet18, DenseNet 201, Inception v3, MobilenetV2
 - Methods:
 - **Random Search:** searches fixed number of random combination from search space
 - **Bayes Optimization:** determines probabilistic relationship between hyperparameter values and objective function, often Gaussian Processes
 - **Optuna:** implements Bayes optimization Tree Structured Parzen Estimator (TPE) algorithm
- ◆ **Tuned Hyperparameters:** Optimizer, Learning Rate, Batch Size, Weight Decay
- ◆ **Test Accuracies**
 - **Efficient NetB2:** Test1 - 99.99%; Test2 - 98.13%
 - **Resnet18:** Test1 - 99.99%, Test2 - 98.16%
 - **DenseNet201:** Test1 - 99.99%; Test2 - 97.23%
 - **Inception v3:** Test1 - 99.99%; Test2: 98.12
 - **MobilenetV2:** Test1 -98.98%; Test 2 - 96.12

→ **Takeaway/Improvement:** We limited tuning to few hyperparameters in our model based on influential hyperparameters seen in previous research. However, since this didn't improve model performance, we can expand both the domain and number of hyperparameters being tuned simultaneously.



Attention

[Task 5] Entropy Regularization

- Main goal: Add regularization to attention scores to avoid them peaking attention scores
 - ◆ A few patches may have very high attention scores, might dominate the low level embeddings of the case

The core mechanism of this change was made in models.py:

- Calculated entropy of attention weights in AttentionPool class
- In the forward() method:

```
# Old
weighted_x = (weights * x).sum(dim=1)
return weighted_x

# New
entropy = -torch.sum(weights * torch.log(weights + 1e-10), dim=1) # Calculate Entropy
entropy_mean = entropy.mean()
weighted_x = (weights * x).sum(dim=1)
return weighted_x, entropy_mean # Return Entropy
```



Attention

Step 2: Capture and aggregate entropy terms in HierarchicalAttnMIL class

- In process_single_stain(), aggregate the entropies for each patch to calculate the average patch entropy for that specific stain

```
# Old
slice_emb = self.patch_attention(patch_embeddings.unsqueeze(0))

# New
slice_emb, patch_H = self.patch_attention(patch_embeddings.unsqueeze(0))
patch_entropies.append(patch_H) # Store for aggregation
```

Step 3: Combine entropies from all 3 levels (Patch, Stain, Case) into a final dictionary for the loss function

- Add lists to track entropy values across all stains in forward()
- Captures entropy from the final case
- Outputs a dictionary containing the averaged entropies for all 3 levels

```
# New Logic added to forward()
final_patch_entropy = torch.stack(all_patch_entropies).mean()
final_stain_entropy = torch.stack(all_stain_entropies).mean()

entropy_reg_terms = {
    'patch': final_patch_entropy,
    'stain': final_stain_entropy,
    'case': case_H
}

return logits, entropy_reg_terms
```



Attention

Additional changes in **trainer.py** and **config.py** :

- Pulls lambda values for patch, stain, and case from newly added parameters in config.py that control the strength of regularization at each level

```
'lambda_patch': 0.0001,  
'lambda_stain': 0.00005,  
'lambda_case': 0.00001,
```

- Created a dictionary to accumulate the entropy values across all batches
- Incorporates the entropy penalty into the final loss function to force the model to learn more uniform weights

```
# Old  
outputs = self.model(stain_slices)  
loss = self.criterion(outputs, label)  
  
# New  
logits, entropy_dict = self.model(stain_slices)  
  
total_entropy_penalty = (l_patch * entropy_dict['patch'] +  
                        l_stain * entropy_dict['stain'] +  
                        l_case * entropy_dict['case'])  
  
loss = self.criterion(logits, label) - total_entropy_penalty
```



Attention

Results:

- Initial run with higher lambdas performed worse than base model
 - ◆ **Patch Level:** Maintained high entropy (~ 3.05), indicating broad, stable attention
 - ◆ **Case Level:** Lower entropy (~ 0.82) shows the model learned specific selectivity
 - ◆ Drop in test accuracy from 77% to 67% suggests current regularization strength might be too aggressive
 - Model might be struggling to focus on the specific peaky features necessary for high grade classification

- Second run with lower lambdas did much better
 - ◆ Recovered model accuracy
 - ◆ Lowering penalty balanced feature selectivity with attention stability

```
'lambda_patch': 0.001,  
'lambda_stain': 0.0005,  
'lambda_case': 0.0001,
```

```
Test Results:  
Test Loss: 1.3220  
Test Accuracy: 0.6667  
Samples: 18
```

```
'lambda_patch': 0.0001,  
'lambda_stain': 0.00005,  
'lambda_case': 0.00001,
```

```
Test Results:  
Test Loss: 0.7878  
Test Accuracy: 0.7778  
Samples: 18
```



Attention

[Task 6] Gated Attention

- Main goal: Current MLP is linear between -1 and 1, gated attention will add more nonlinearity to this model

Moderate changes to weight calculation in the **AttentionPool** class in models.py:

→ Adds 2 parallel paths in init():

- ◆ tanh for feature transformation
- ◆ sigmoid for gating mechanism

```
self.attention_v = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),
    nn.Tanh()
)

# Gating mechanism (sigmoid)
self.attention_u = nn.Sequential(
    nn.Linear(input_dim, hidden_dim),
    nn.Sigmoid()
)

# Final projection to a single score
self.attention_weights = nn.Sequential(
    nn.Dropout(dropout),
    nn.Linear(hidden_dim, 1),
)
```

→ Calculate gated attention weights in forward():

```
v = self.attention_v(x) # (B, M, H)
u = self.attention_u(x) # (B, M, H)
gated = v * u # (B, M, H)

# Compute final attention scores
weights = self.attention_weights(gated)
weights = torch.softmax(weights, dim=1)

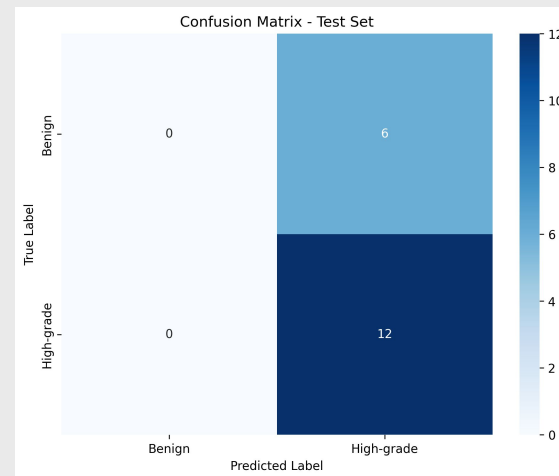
weighted_x = (weights * x).sum(dim=1) #
```



Attention

Results

- Model blindly predicted "High Grade" for every case
(12/18) = 0.6667
 - ◆ Likely collapsed to predicting only the majority class; failed to find a signal that generalized to these 18 new cases
- Model performed well in validation loop
 - ◆ Peak validation accuracy: 88.24%
 - ◆ Best validation loss: 0.4737
 - ◆ Might be overfitting → gated model memorized the features of the validation dataset that did not exist in the test set
- Next steps:
 - ◆ Finding a way to regularize this model might improve the accuracy
 - Gated attention + entropy regularization might work



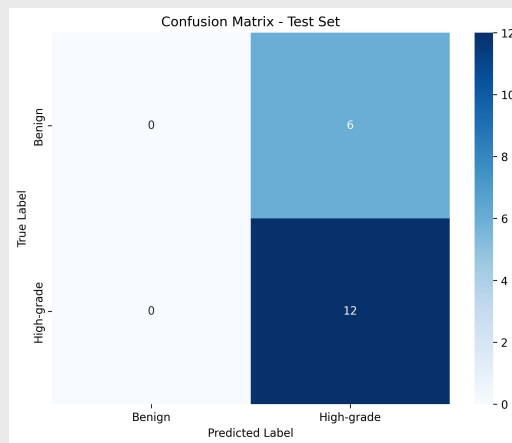
Test Results:
Test Loss: 1.6279
Test Accuracy: 0.6667
Samples: 18



Attention

[Task 7] Interaction-aware Attention

- Attempted along with gated attention but results are the same as before
- Ran out of time to figure out the best way to implement this, might work with further investigation and without gated attention



Test Results:
Test Loss: 0.6425
Test Accuracy: 0.6667
Samples: 18



Regularization (Dropout)

→ What is Dropout?

Zeroes out random selection of neurons (data points) to prevent co-adaptation.

- **Co-adaptation:** Neurons highly dependent on one another, increases risk of overfitting.

Forces the model to learn the hidden features better.

Standard dropout, defined with one parameter, **p** (or **rate**).

- **p** neurons are "zeroed out" (removed from consideration)
- Remaining neurons are re-scaled, overall sum of neuron values remains the same

$$x_i = \frac{1}{1-p} * x_i$$

- Current model uses fixed dropout rate of **0.3** (set in **config.py**)
 - 30% of neurons are zeroed out
 - Remaining neurons multiplied by ~1.4286 to maintain sum



Regularization (Dropout)

→ Three stages: patch projection, attention score, classifier

- In the original model, dropout rate is consistent across three stages ($p = 0.3$ for all)
- Network width shrinks steadily as it deepens

→ Plan to optimize Dropout

If we continue to use Standard Dropout, next step is tuning the dropout rate for each stage as separate parameter

- Why? It's possible that optimal rates differ across depths in model
- Note: We didn't tune these parameters for Presentation 2 because it was too computationally expensive, i.e. training would have taken too long (pushed to Presentation 3).

Alternatively, we could deviate from Standard Dropout...

d_1 = Dropout rate for patch projection

d_2 = Dropout rate for attention scores

d_3 = Dropout rate for classifier



Regularization (Dropout)

→ Proposal: 2D Spatial Dropout

What	Drops entire 2D feature maps, not individual elements (like standard Dropout).
How	Input a 4D tensor. If high correlation between adjacent pixels, then the entire channel (instead of the single element) is zeroed.
Why	Dropout used to prevent the model from relying too heavily on certain features and overfitting. But with images, removing a few elements, i.e. pixels, won't prevent the model from getting a sense of the shape.
When	Generally more effective in earlier stages of CNNs, will be most effective in the patch projection stage.

→ Alternate proposal: Cutout

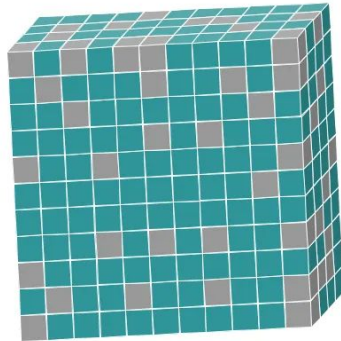
Like Spatial Dropout, Cutout removes larger sections of the image, but randomly. Random distribution of removed patches prevents model from overfitting using one portion.



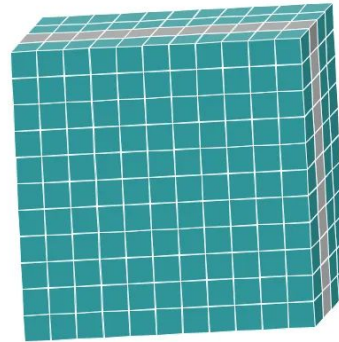
Regularization (Dropout)

→ Different dropout methods, visualized

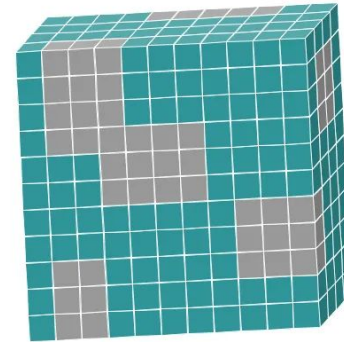
Standard Dropout



Spatial Dropout



Cutout





Regularization (Attention Scores)

→ **Proposal: Entropy-based Attention Regularization (EAR)**

Let a be the series of attention weights (normalized attention scores, post-softmax) for a given case, patch, or slice (we could apply this at all three layers).

$$a = (a_1, a_2, \dots, a_N)$$

$$a_i \geq 0$$

$$\sum_{i=1}^N a_i = 1$$

Then, we calculate the EAR term as

$$H(a) = - \sum_{i=1}^N a_i \log a_i$$

And add this into the existing loss function (binary cross-entropy)

$$L_{\text{total}} = L_{\text{BCE}} + H(a)$$

(Attanasio et. al, 2022; Yan et. al, 2022)



Regularization (Weight Decay)

→ How does weight decay work?

- Reduces overfitting by adding a penalty term (L_2 norm) to the loss function (see below)
- Penalizes large weights

$$L_{\text{total}} = L_{\text{BCE}} + \lambda \sum_{i=1}^N W_i^2$$

→ Last quarter

- Increased **weight_decay** from **0.00001** to **0.0002**
- Harsher punishment for large coefficients → reduces overfitting

→ Proposal: Tune weight decay (λ)

- Initial model appears to be **underfitting** the data
- Sofie's optimal value (from Optuna tuning): 1.1528×10^{-6}
 - **Decrease** from previous value, suggests that lower penalty is necessary



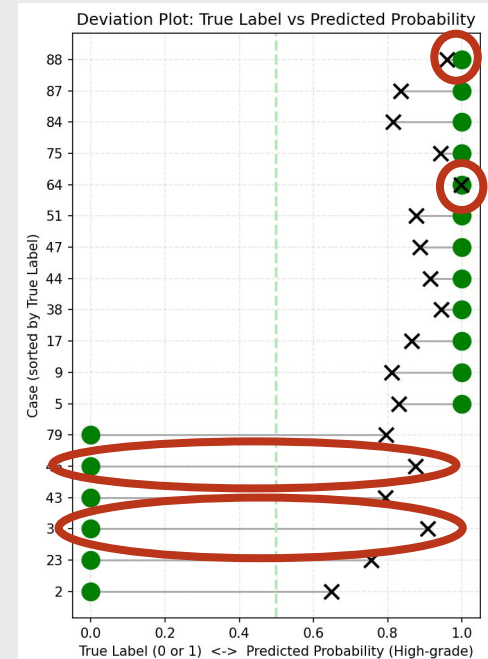
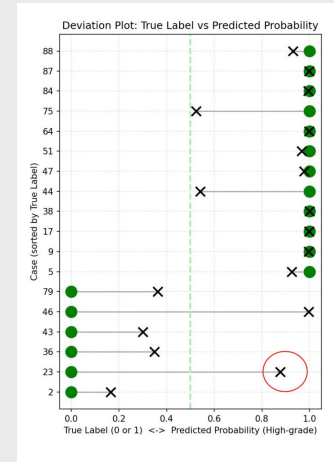
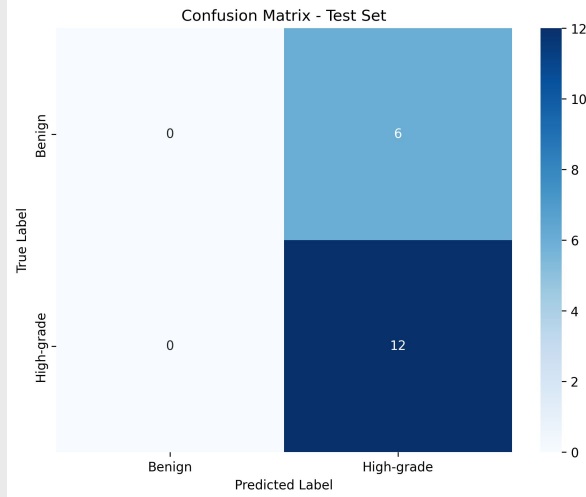
Results

Only going to analyze Optuna tuned model to observe how it altered behavior of model performance

Performance:

Test Loss: .6395 (-.0877), Test Accuracy: 66.7% (-11.1%)

Validation Loss: .6161, Validation Accuracy: 70.59%



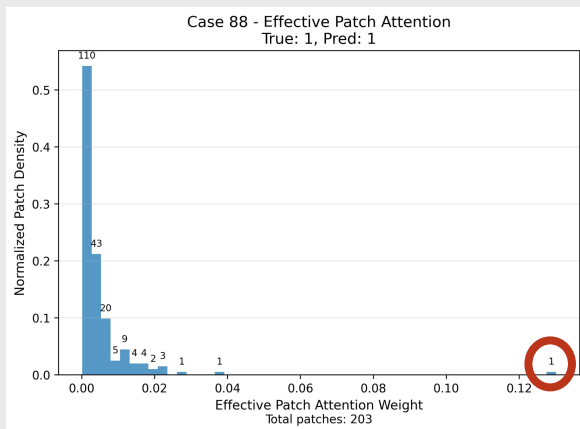


Results - Most Accurate

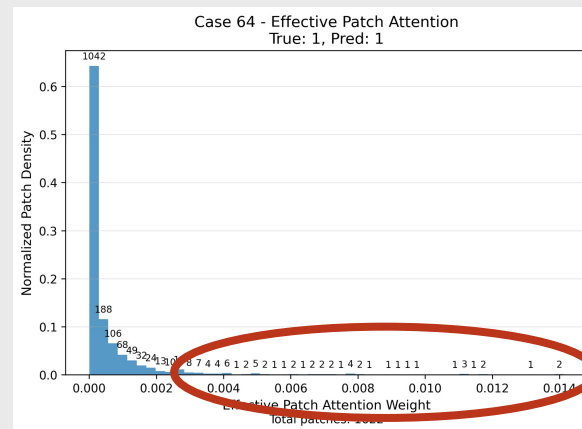
Aggressively predicted high-grade for all, making loss smaller because more accurate, yet errors for benign much worse hence overall worse accuracy

Appears to be the case that **in high-grade, there is one patch** in particular that very obviously **conveys presence of cancer**

Case 88:



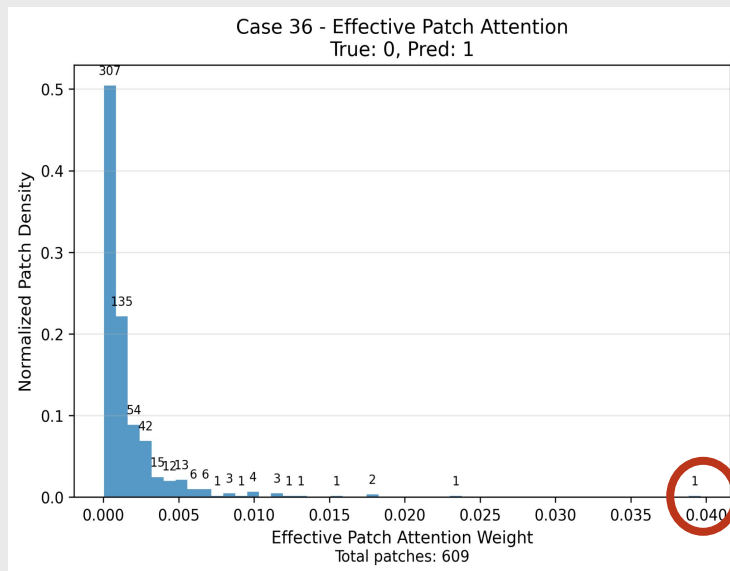
Case 64:



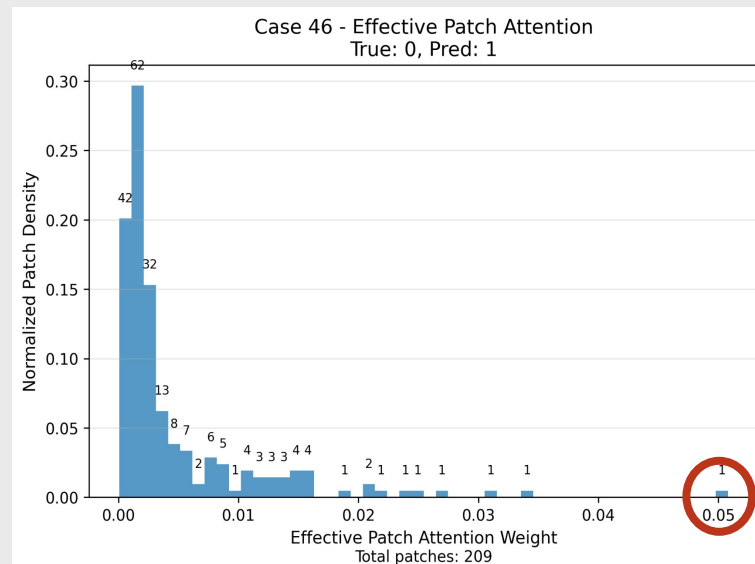


Results - Least Accurate

Case 36:



Case 46:

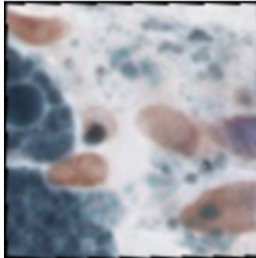




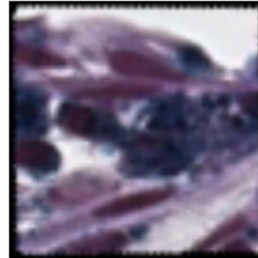
Results

Case 88 - Top Effective Patches

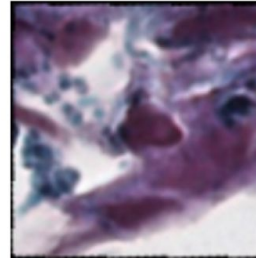
sox10 s0 p48
Eff: 0.1301



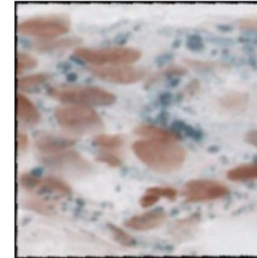
h&e s1 p5
Eff: 0.0386



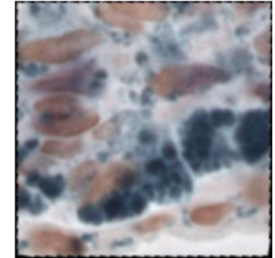
h&e s0 p40
Eff: 0.0272



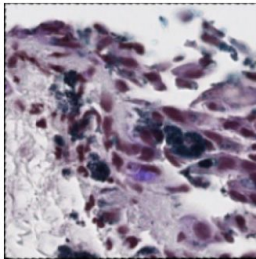
sox10 s0 p20
Eff: 0.0233



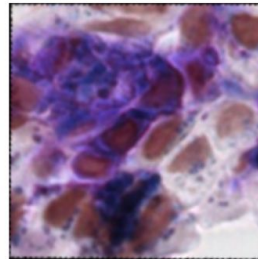
sox10 s0 p32
Eff: 0.0215



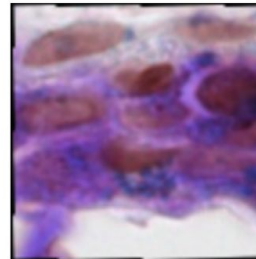
h&e s2 p16
Eff: 0.0210



melan s0 p46
Eff: 0.0199



melan s0 p28
Eff: 0.0184

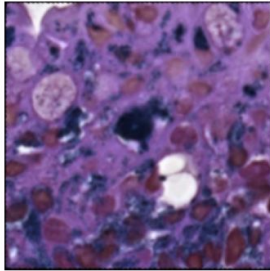




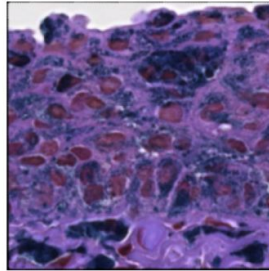
Results

Case 64 - Top Effective Patches

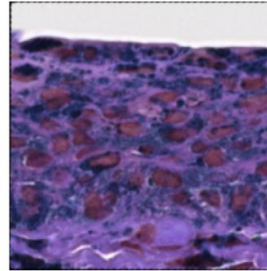
h&e s3 p27
Eff: 0.0141



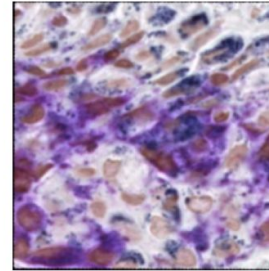
h&e s1 p39
Eff: 0.0140



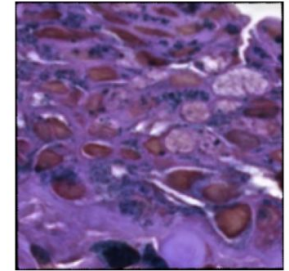
h&e s1 p58
Eff: 0.0131



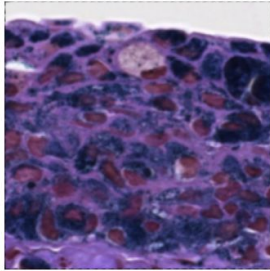
melan s3 p63
Eff: 0.0118



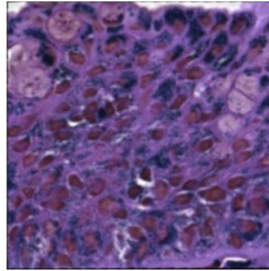
h&e s3 p24
Eff: 0.0117



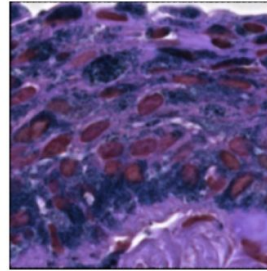
h&e s1 p47
Eff: 0.0114



h&e s2 p30
Eff: 0.0113



h&e s1 p66
Eff: 0.0112

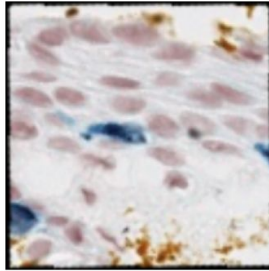




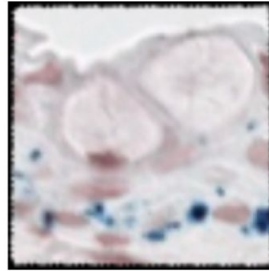
Results

Case 36 - Top Effective Patches

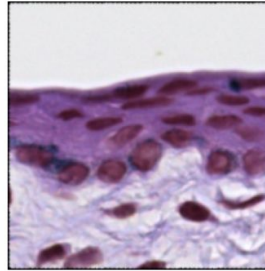
melan s0 p41
Eff: 0.0396



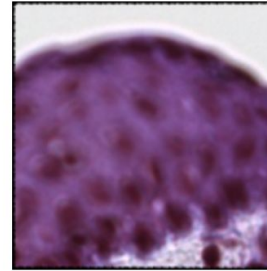
melan s1 p21
Eff: 0.0236



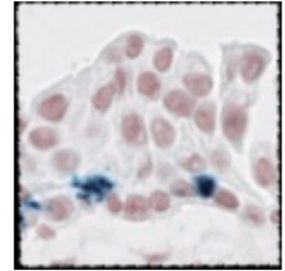
h&e s3 p1
Eff: 0.0180



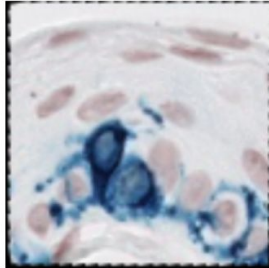
h&e s2 p36
Eff: 0.0180



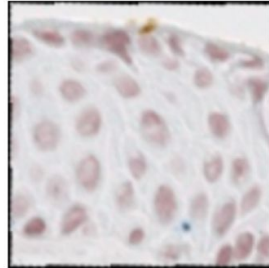
melan s0 p28
Eff: 0.0156



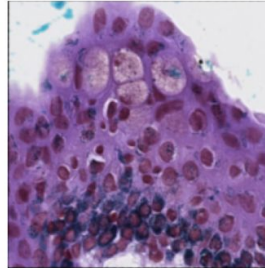
melan s0 p13
Eff: 0.0129



melan s1 p5
Eff: 0.0127



h&e s1 p26
Eff: 0.0117

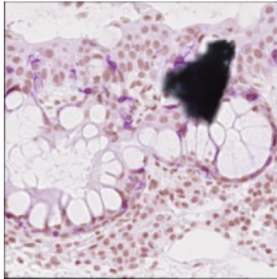




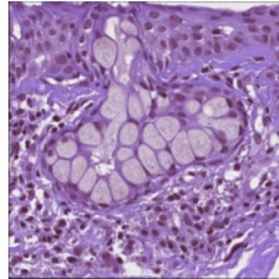
Results

Case 46 - Top Effective Patches

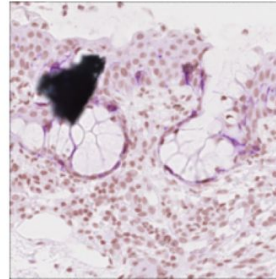
melan s0 p29
Eff: 0.0508



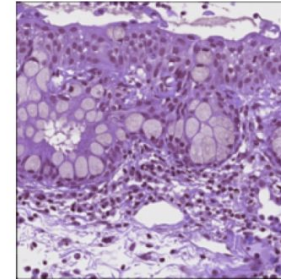
h&e s0 p49
Eff: 0.0338



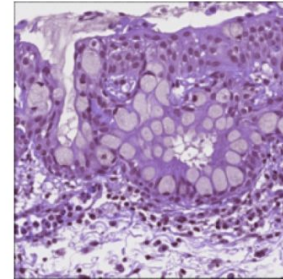
melan s0 p54
Eff: 0.0311



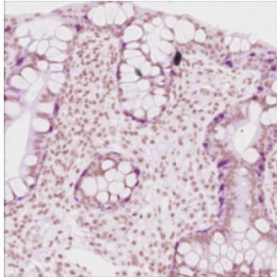
h&e s0 p13
Eff: 0.0272



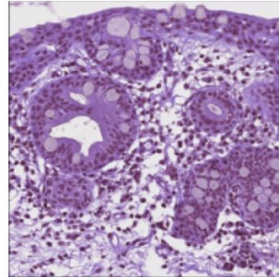
h&e s0 p19
Eff: 0.0250



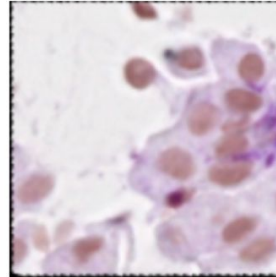
melan s0 p64
Eff: 0.0243



h&e s0 p6
Eff: 0.0222



melan s0 p21
Eff: 0.0210





Results

Case 88:

Most attended stain: sox10

Stain-level attention:

h&e: 0.3140

melan: 0.3224

sox10: 0.3636

Case 36:

Most attended stain: melan

Stain-level attention:

h&e: 0.3607

melan: 0.3608

sox10: 0.2786

Case 64:

Most attended stain: h&e

Stain-level attention:

h&e: 0.5866

melan: 0.3010

sox10: 0.1123

Case 46:

Most attended stain: h&e

Stain-level attention:

h&e: 0.4125

melan: 0.3700

sox10: 0.2175



Takeaways

Lack of improvement across all tested strategies suggest a **pivot in next approach**

- **Hyperparameter Tuning** - underperformed despite expectations to pick more suitable parameters
 - ◆ Individually tune dropout in each of its implementations (attention scores vs. classifiers, etc.)
 - ◆ Add more trials, n_warmup_step, epochs to allow model to learn more extensively
 - ◆ Tune more hyperparameters
- **Dropout** - deviate from standard dropout and explore 2D Spatial or Cutout
 - ◆ Remove highly correlated sections as opposed to individual pixels to reap benefits of dropout
 - ◆ Consider implementing in "patch dropout stage"
- **Attention Scores** - model struggling to focus on the specific peaky features
 - ◆ Smaller penalty/lambda values perform better
 - ◆ Look for solutions at the patch level given attention score distribution
 - ◆ Gated Attention is too aggressive – predicts everything as high-grade, potentially try regularizing



References

Watanabe, Shuhei. "Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance". *arXiv*, arXiv:2304.1127. 2025. <https://arxiv.org/abs/2304.1127>

Wasito, Ito and Denny Saptono F." Deep Transfer Learning Base Models with Optuna Hyperparameter and Optimization on Colorectal Cancer Images Classification". *International Journal of Advanced Research*. June 2025. pp 1638-1648. <https://dx.doi.org/10.21474/IJAR01/21230>

Thank you!